PythonDay3

March 16, 2026

1 Zakipoint Health Internship – Day 3

Name: Dipendra Raj Bhatt

1.1 1. Python Programming

1.1.1 1.0.1 Python Fundamentals

- Variables
- Loops
- Functions
- Methods

1.1.2 1.2 Logical Thinking & Coding Problems

- Practice solving algorithmic problems
- Break down problems into smaller steps

1.1.3 1.3 Object-Oriented Programming (OOP)

- Classes
- Inheritance
- Polymorphism
- Magic Methods

1.1.4 1.4 Decorators and Generators

- Learn function decorators
- Use generators for memory-efficient loops

1.1.5 1.5 Exception Handling & Logging

- Try/Except blocks
- Logging best practices

1.1.6 1.6 File Handling

- Reading & writing files
- Working with CSV, JSON

1.1.7 1.7 Writing Clean, Modular Code

- Functions and modules
- Code readability and PEP8

1.2 1.4 Decorators and Generators

```
Learn function decorators  |
Use generators for memory-efficient loops
```

2 Decorators

A **decorator** is a function that takes another function as an argument and returns a modified function.

1. Used to **extend function behavior without modifying the original function**.
2. Applied using the **`@decorator_name` syntax**.
3. Uses a **wrapper function** inside the decorator.
4. Often uses **`*args` and `**kwargs`** to support any parameters.
5. Common in **logging, authentication, caching, and timing functions**.

Syntax "'python def decorator_name(function_name): def wrapper(*args, **kwargs): # additional functionality result = function_name(*args, **kwargs) # additional functionality return result return wrapper

@decorator_name def function_name(parameters): pass

2.1 Generators

A **generator** is a function that returns an iterator and produces values **one at a time** using the `yield` keyword.

1. Uses **`yield`** instead of `return`.
2. Gives values **one by one**, only when needed.
3. The function **stops at `yield`** and continues from there next time.
4. Uses **less memory** because it generates values one by one.
5. You can get values using a **`for` loop** or the **`next()` function**.

Syntax "'python def generator_name(parameters): # initialization while condition: yield value

[1]:
```
def decorator(function):
    def wrapper():
        print("Transaction Initiated")
        function()
        print("Transaction Completed")

    return wrapper

def hello():
    print("Executing all steps of transaction")

dec = decorator(hello)
dec()
```

```
Transaction Initiated
Executing all steps of transaction
Transaction Completed
```

[2]:
```
# Generators

def creater():
    i=1
    while i<= 200:

        yield i
        i+=1

print(creater())

X= creater()
print(next(X))
print(next(X))

print(list(X))
```

```
<generator object creater at 0x7232ba54b4c0>
1
2
[3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16, 17, 18, 19, 20, 21, 22, 23,
24, 25, 26, 27, 28, 29, 30, 31, 32, 33, 34, 35, 36, 37, 38, 39, 40, 41, 42, 43,
44, 45, 46, 47, 48, 49, 50, 51, 52, 53, 54, 55, 56, 57, 58, 59, 60, 61, 62, 63,
64, 65, 66, 67, 68, 69, 70, 71, 72, 73, 74, 75, 76, 77, 78, 79, 80, 81, 82, 83,
84, 85, 86, 87, 88, 89, 90, 91, 92, 93, 94, 95, 96, 97, 98, 99, 100, 101, 102,
103, 104, 105, 106, 107, 108, 109, 110, 111, 112, 113, 114, 115, 116, 117, 118,
119, 120, 121, 122, 123, 124, 125, 126, 127, 128, 129, 130, 131, 132, 133, 134,
135, 136, 137, 138, 139, 140, 141, 142, 143, 144, 145, 146, 147, 148, 149, 150,
151, 152, 153, 154, 155, 156, 157, 158, 159, 160, 161, 162, 163, 164, 165, 166,
167, 168, 169, 170, 171, 172, 173, 174, 175, 176, 177, 178, 179, 180, 181, 182,
```

```
183, 184, 185, 186, 187, 188, 189, 190, 191, 192, 193, 194, 195, 196, 197, 198,
199, 200]
```

[ ]:

[ ]: